

Poprawiliśmy realny błąd w kodzie. GPT zdekodował o 24 punkty procentowe gorzej.

Test poprawki „zanieczyszczenia kontekstu” na 4 modelach LLM (N=100 każdy, wariant zero-shot) — czy czystszy zestaw ramek historycznych poprawia rozpoznawanie sygnałów w nieznanymi ramkach CAN?

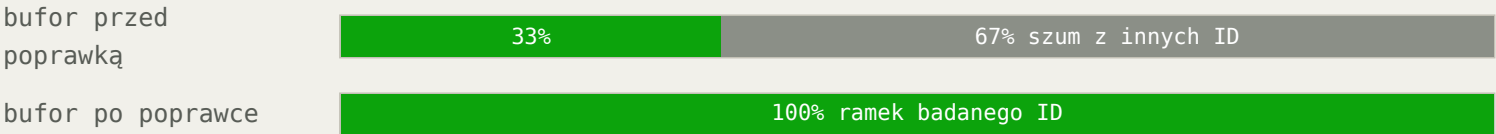
HIPOTEZA WYJŚCIOWA

Czystszy kontekst (ramki tylko z badanego CAN ID, zamiast wymieszanych z 3 ID) poprawi rozpoznawanie flag bitowych — najłabszej kategorii sygnałów w Eksperymentie 4.1.

OBALONA

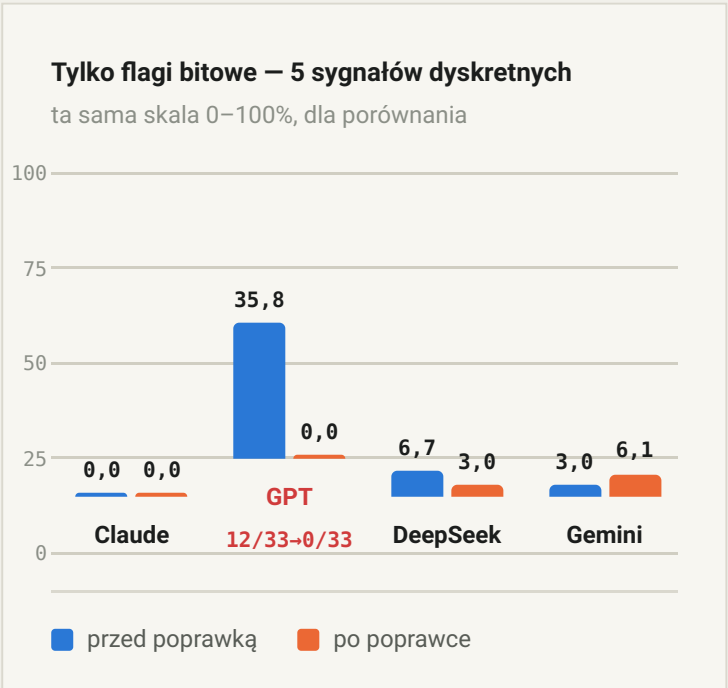
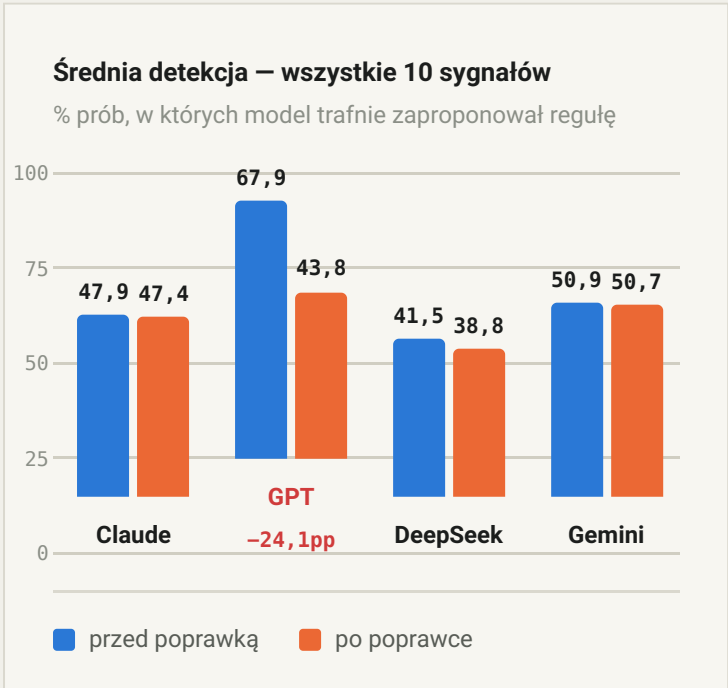
CZYM BYŁO ZANIECZYSZCZENIE KONTEKSTU

Prompt systemowy obiecuje modelowi „recent frames FOR THIS ID”. Przed poprawką wszystkie 3 badane CAN ID współdzieliły jeden bufor — po zapelnieniu okna (30 ramek) tylko część z nich faktycznie należała do pytanego ID:



Poprawka: QHash<canId, deque<CanFrame>> — osobny bufor 30 ramek na każde CAN ID, zamiast jednego współdzielonego.

WYNIK: ŚREDNIA DETEKcja (10 SYGNAŁÓW) I SYGNAŁY Dyskretne (5 FLAG BITOWYCH)



POPRAWKA KODU ≠ POPRAWA WYNIKU MODELU

Naprawa **nie poprawiła** wyników żadnego z 4 modeli. U GPT spowodowała **drastyczną regresję** — spadek z 67,9% do 43,8% ogółem, a próby dekompozycji bitowej na CAN ID 0x200 spadły z 12/33 do dosłownie 0/33.

Robocza hipoteza: flagi bitowe przełączają się rzadko (co 3–20s) względem cyklu odpytywania 3 CAN ID. Czysty bufor bywa więc mało zróżnicowany (te same wartości bajtu powtórzone) — model rzadziej „widzi” moment przełączenia. Stary, zanieczyszczony bufor przypadkiem pokazywał więcej pozornej różnorodności (choć błędnie przypisanej z innych ID), co mogło częściej podpowiadać hipotezę o złożonej strukturze bajtu.

To już **czwarty z rzędu** negatywny/neutralny wynik interwencji promptu/kontekstu wobec tego samego problemu (zero-shot, few-shot, entropy-analysis, context-fix) — silny sygnał, że modele zawodnie liczą entropię bajtu z surowego hexu, niezależnie od instrukcji czy jakości danych. Poprawka mimo to zostaje w kodzie — jest słuszną architektonicznie, niezależnie od tego, że akurat nie pomogła w tym zadaniu.

→ REKOMENDOWANE DALSZE KROKI

1 Eksperyment 4.2 – statystyki per bajt w promptcie badawczy

Zamiast prosić model, żeby sam policzył entropię/niesekwencyjność bajtu z surowego hexu (4 dotychczasowe próby to pokazały jako zawodne), dostarczamy te statystyki JUŻ POLICZONE (deterministycznie, w C++). Zmienia zadanie z „policz i zauważ” na „zinterpretuj gotową obserwację”. Szczegóły: `Eksperyment_4.2_Propozycja_Dalszej_Optymalizacji_LLM_20260728.md`

2 Hybrydowy override klasyczny produkcyjny, wdrażalny od razu

Jeśli LLM zaproponuje skalar, kod deterministycznie sprawdza, czy wartości tworzą wzorec typowy dla flag bitowych, i programowo wymusza dekompozycję. Gwarantowana poprawa, niezależna od ograniczeń modelu — nie wymaga dalszych eksperymentów promptowych.